

A breakdown of the computation needed for this approach can be found in Appendix B.

4.1.1 Suffix Arrays

This exact-substring-matching criterion, while conceptually simple, is computationally prohibitive with naive (quadratic) all-pair matching. To improve the efficiency, we concatenate all the examples of the entire dataset D into a giant sequence $\mathcal{S}$, and construct a Suffix Array $\mathcal{A}$ of $\mathcal{S}$. A suffix array (Manber and Myers, 1993) is a representation of a suffix tree (Weiner, 1973) that can be constructed in linear time in $\|\mathcal{S}\|$ (Kärkkäinen and Sanders, 2003) and enables efficient computation of many substring queries; in particular, they allow us to identify duplicated training examples in linear time. Suffix arrays have the advantage over suffix trees in that they are 10–100 $\times$ more memory efficient (Manber and Myers, 1993), requiring just 8 bytes per input token, though they are asymptotically less efficient for some query types. They have been used widely in NLP, such as for efficient TF-IDF computation (Yamamoto and Church, 2001) and document clustering (Chim and Deng, 2007).

The suffix array $\mathcal{A}$ for a sequence $\mathcal{S}$ is a lexicographically-ordered list of all suffixes contained in the sequence. Formally,

$$\mathcal{A}(\mathcal{S}) = \arg \text{sort all_suffixes}(\mathcal{S})$$

For example, the suffixes of the sequence “banana” are (“banana”, “anana”, “nana”, “ana”, “na”, “a”) and so the suffix array is the sequence (6 4 2 1 5 3). In practice, we construct $\mathcal{S}$ from the bytes of the BPE tokenization of the text (§6).

4.1.2 Substring matching

After constructing $\mathcal{A}$, it is straightforward to identify duplicated training examples. Suppose that the sequence s was repeated exactly twice in the training dataset $\mathcal{S}$ at positions i and j , that is, $\mathcal{S}_{i..i+|s|} = \mathcal{S}_{j..j+|s|}$. Then the indices i, j will occur adjacent to each other in the suffix array $\mathcal{A}$.

Finding all repeated sequences is thus a matter of linearly scanning the suffix array from beginning to end and looking for sequences $\mathcal{A}_i, \mathcal{A}_{i+1}$ that share a common prefix of at least some threshold length. Any satisfying sequences are recorded. This algorithm is embarrassingly parallel, and so we can efficiently process the dataset. Based on experimentation (Appendix B), we choose a threshold length of 50 BPE tokens for all experiments.

4.2 Approximate Matching with MinHash

We also perform *approximate* deduplication based on matching entire examples. This method, which we call NEARDUP, is a good complement to the *exact* substring matching, especially for web crawl text, as it handles the very common case of documents being identical except for interspersed templated fields (such as the last row of Table 1).

MinHash (Broder, 1997) is an approximate matching algorithm widely used in large-scale deduplication tasks (Versley and Panchenko, 2012; Gabriel et al., 2018; Gyawali et al., 2020), including to deduplicate the training set for a large Chinese-language LM (Zeng et al., 2021). Given two documents x_i and x_j , the main idea is to represent each document by its respective set of n -grams d_i and d_j . We can then use hash functions to approximate the *Jaccard Index* (Jaccard, 1912):

$$\text{Jaccard}(d_i, d_j) = |d_i \cap d_j| / |d_i \cup d_j|$$

If the Jaccard Index between d_i and d_j is sufficiently high, it is likely that documents are approximate matches of each other. To efficiently approximate the Jaccard index, MinHash constructs document signatures by sorting each of the n -grams via a hash function, and then keeping only the k smallest hashed n -grams. There are multiple ways to construct estimators of the Jaccard index from these kinds of signatures (Cohen, 2016).

In our implementation, we use 5-grams and a signature of size 9,000. The probability that two documents are considered a potential match is

$$\Pr(d_i, d_j | \text{Jaccard}(d_i, d_j) = s_{i,j}) = 1 - (1 - s_{i,j}^b)^r$$

where $b = 20$ and $r = 450$ are user-settable parameters to control the strength of the filter. See Appendix A for more details.

For each pair of documents identified as a potential match, more computationally expensive similarity metrics can be employed as a subsequent filtering step. In particular, we identify two documents as duplicates if they are matched by the MinHash algorithm and their *edit similarity* is greater than 0.8. The edit similarity between token sequences x_i and x_j is defined as:

$$\text{EditSim}(x_i, x_j) = 1 - \frac{\text{EditDistance}(x_i, x_j)}{\max(|x_i|, |x_j|)}$$

To build clusters of similar documents, we construct a graph that has an edge between two documents if they are considered a match. Then, we

Dataset	Example	Near-Duplicate Example
Wiki-40B	\n_START_ARTICLE\nHum Award for Most Impactful Character \n_START_SECTION\nWinners and nominees\n_START_PARAGRAPH\nIn the list below, winners are listed first in the colored row, followed by the other nominees. [...]	\n_START_ARTICLE\nHum Award for Best Actor in a Negative Role \n_START_SECTION\nWinners and nominees\n_START_PARAGRAPH\nIn the list below, winners are listed first in the colored row, followed by the other nominees. [...]
LM1B	I left for California in 1979 and tracked Cleveland's changes on trips back to visit my sisters .	I left for California in 1979 , and tracked Cleveland's changes on trips back to visit my sisters .
C4	Affordable and convenient holiday flights take off from your departure country, "Canada". From May 2019 to October 2019, Condor flights to your dream destination will be roughly 6 a week! Book your Halifax (YHZ) - Basel (BSL) flight now, and look forward to your "Switzerland" destination!	Affordable and convenient holiday flights take off from your departure country, "USA". From April 2019 to October 2019, Condor flights to your dream destination will be roughly 7 a week! Book your Maui Kahului (OGG) - Dubrovnik (DBV) flight now, and look forward to your "Croatia" destination!

Table 1: Qualitative examples of near-duplicates identified by NEARDUP from each dataset. The similarity between documents is highlighted. Note the small interspersed differences that make exact duplicate matching less effective. Examples ending with “[...]” have been truncated for brevity. More data available in Appendix.

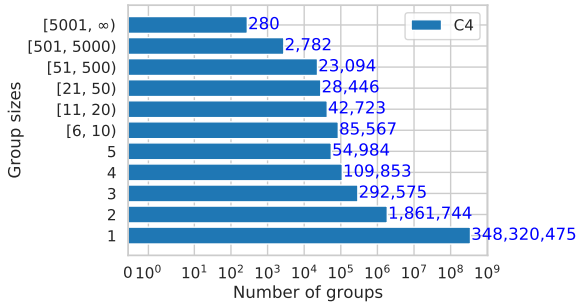


Figure 1: The distribution of near-duplicate cluster sizes from running NEARDUP on C4.

use the method introduced in Łącki et al. (2018) to identify connected components. A breakdown of the computation needed is given in Appendix A.

5 Deduplication Results

We deduplicate each of the four datasets with both of our two techniques. When text was duplicated across multiple data splits, we prioritized keeping a copy in the test or validation set and removing it from the train set.

5.1 Amount of Text Removed

With NEARDUP, we found that the web-scrape datasets contain between 3.04% (on C4) to 13.63% (on RealNews) near duplicates (Table 2). Near-duplicate text is much less common in Wiki-40B, forming only 0.39% of the train set.² In C4, the majority (1.8M) of near-duplicate clusters consisted of just a single pair of examples that matched against each other, but there were 280 clusters with over 5,000 examples in them (Figure 1), including one cluster of size 250,933.

²Most duplicates we saw were automatically generated pages, such as the outcomes of sports games. This shows the strength of manual curation for creating high-quality datasets.

	% train examples with dup in train	% valid with dup in valid	% valid with dup in train
C4	3.04%	1.59%	4.60%
RealNews	13.63%	1.25%	14.35%
LM1B	4.86%	0.07%	4.92%
Wiki40B	0.39%	0.26%	0.72%

Table 2: The fraction of examples identified by NEARDUP as near-duplicates.

	% train tokens with dup in train	% valid with dup in valid	% valid with dup in train
C4	7.18%	0.75 %	1.38 %
RealNews	19.4 %	2.61 %	3.37 %
LM1B	0.76%	0.016%	0.019%
Wiki40B	2.76%	0.52 %	0.67 %

Table 3: The fraction of tokens (note Table 2 reports the fraction of *examples*) identified by EXACTSUBSTR as part of an exact duplicate 50-token substring.

On average with EXACTSUBSTR, we remove more total content than with NEARDUP (despite EXACTSUBSTR not removing any examples outright)—for example removing 7.18% of the tokens in C4. The exception is LM1B, where EXACTSUBSTR removes 8× less data than NEARDUP. On investigation, we find this is due to the fact that LM1B documents are significantly shorter: 90% of all documents are under 50 tokens, and so are not even candidates for potential matches even if the entire sequence matched verbatim. We find that both NEARDUP and EXACTSUBSTR remove similar content—77% of the training examples that NEARDUP removes from C4 have at least one verbatim length-50 match found by EXACTSUBSTR.

5.2 Properties of Duplicated Text

While the authors of both RealNews and C4 explicitly attempted deduplication during dataset construction, the methods were insufficient to capture the more subtle types of duplicate text commonly found on the internet. In C4 and Wiki-40B, we qualitatively observe that much of the text identified as near-duplicated is computer-generated. The text is identical except for the names of places, businesses, products, dates, and so on. Because these examples frequently differ by just a few words at a time, deduplication strategies relying on exact string matching would fail to identify a match. Example duplicate pairs from each dataset can be found in Table 1 (more examples in the Appendix).

For RealNews and LM1B, derived from news sites, we observe that many near-duplicates occur because the same news article appears on multiple news sites with slightly different formatting. For example, in LM1B, there is one example that starts “*MINEOLA , N.Y. - New York officials say [...]*” and another that starts “*(AP) - New York officials say [...]*”. The two examples are otherwise identical.

5.3 Train / Test Set Leakage

Both deduplication methods identify overlap between the train set and the validation set (Table 2). For example, 4.6% of the C4 validation set and 14.4% of the RealNews validation set examples had an approximate duplicate in their respective training sets. Such duplication is problematic since it could cause evaluation metrics to be unfairly inflated for models that are better at memorizing their train sets. We evaluate the effect of this leakage on publicly released models in Section 6.3.

6 Impact on Trained Models

We trained 1.5B parameter “XL”, decoder-only, Transformer-based language models similar to GPT-2, on C4-ORIGINAL, C4-NEARDUP, and C4-EXACTSUBSTR, respectively. We use the T5 codebase and model architecture from Raffel et al. (2020), and each model was trained for about two epochs on its respective dataset. To better understand the amount of variance in the perplexities of trained models, we also trained three different random seeds of the 110M parameter “base” model for each of the above three datasets—for a total of nine base-sized models.

For all experiments, we used a Byte Pair Encoding (BPE) vocabulary trained on C4-NEARDUP

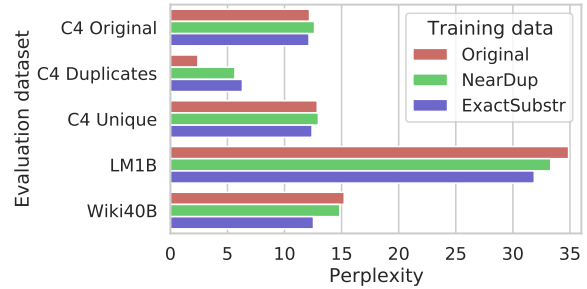


Figure 2: Impact of deduplicating the training set on validation perplexity. We plot the results from T5 XL (see Appendix for base-sized model). For C4, we evaluate on *C4 Original*, the original validation set; *C4 Unique*, a subset of the validation set identified by NEARDUP as having zero matches across C4; and *C4 Duplicates*, a subset of the validation set identified by NEARDUP as having a match in the C4 train set.

with a budget of 50K tokens, which resulted in a vocabulary the same size as GPT-2’s. We trained with a maximum sequence length of 512 tokens (for longer documents, we randomly extracted subsequences of this length.) Further training details can be found in Appendix C.

6.1 Model Perplexity

We computed the perplexity of our trained models on the validation sets of LM1B and Wiki-40B, and on subsets of the C4 validation set (Figure 2). For the base size, we observe that all models have similar perplexity on the original C4 validation set and on validation set examples that were identified as unique (no near-duplicate in either train or validation). However, both models trained on deduplicated data have significantly higher perplexity on validation set examples that have duplicates in the training set than the model trained on the original C4. EXACTSUBSTR-deduplicated results in higher perplexity than NEARDUP-deduplicated. These trends holds true for the XL sized model as well. While this may suggest EXACTSUBSTR deduplication results in models least overfit on the train set, note that both of these techniques have used separate duplicate thresholds and a different choice of thresholds could change the results.

When evaluating on the validation sets of LM1B and Wiki-40B, we found that models trained on NEARDUP-deduplicated C4 consistently achieved lowest perplexity (for LM1B eval with base models, see Appendix Figure 7). EXACTSUBSTR deduplication decreases perplexity of the XL model by almost 3 points perplexity on Wiki-40B which is